

How Fiber Data Structure Works in React

1. What is Fiber?

Fiber is React's internal data structure for representing the component tree.

Before Fiber, React used a recursive tree. With Fiber, React uses a **linked list based tree of nodes**, where each node is a *Fiber*.

Each Fiber = one unit of work.

One Fiber node = one React element + its state + what to do next

2. What does a Fiber node contain?

Each Fiber has links to others, like a graph:

```
fiber = {
  type,
  key,
  stateNode,
  child,
  sibling,
  return,
  pendingProps,
  memoizedProps,
  memoizedState,
  updateQueue
}
```

So instead of deep recursion, React walks this structure step by step.

3. How Fiber works during rendering

Fiber enables **incremental rendering**.

That means React can:

- Pause work
- Resume later
- Abort work
- Reuse work

Step-by-step flow

1. Render phase (Reconciliation)

- React builds a new Fiber tree (work in progress)

- Compares it with the old tree
- Figures out what changed

2. Commit phase

- Applies changes to the DOM
- Runs effects like `useEffect`

4. Why Fiber is powerful

Fiber allows:

- Time slicing → React can stop work to handle user input
- Priority updates → Animations > data fetch updates
- Concurrent rendering → Work can happen in chunks

5. How traversal works (child → sibling → parent)

Instead of recursion:

```
<div>
  <A />
  <B />
</div>
```

Fiber links like:

```
divFiber
  ↓ child
  A Fiber → sibling → B Fiber
```

Traversal order:

1. Go to child
2. If no child, go to sibling
3. If no sibling, go up to parent and try again

6. Simple mental model

Imagine Fiber as:

A to-do list for React where each task is small and interruptible.

Each Fiber node = one task.

7. When you feel it as a developer

You don't use Fiber directly, but you see its effects when you use:

- `useTransition()`
- `Suspense`
- Concurrent features in React 18

They all depend on Fiber under the hood.